

SkillSwap - Project Report

1. Problem Statement

Traditional learning often requires financial resources, making education inaccessible to many. Furthermore, individuals possess valuable skills but lack a structured platform to monetize or trade them directly without relying on traditional currency. This creates an artificial barrier for continuous, accessible, and reciprocal education.

2. Abstract

SkillSwap is a real-time skill barter platform that allows users to exchange their knowledge without monetary transactions. By utilizing a 'Skill Credits' economy, users earn credits by teaching others and spend those credits when they want to learn. The platform integrates an AI matching system using cosine similarity to intelligently connect users based on complementary skills, their geographical availability, and their overall reputation score.

3. Proposed Solution

The proposed solution is a full-stack, distributed web and mobile application. It combines a highly responsive, modern React frontend (deployed on the Web and packaged natively for Android via Capacitor) with a robust Python Django backend. To ensure high-quality exchanges, the system features an AI-driven matchmaking algorithm, real-time WebSocket chat for seamless session coordination, and a comprehensive multi-factor reputation system.

4. System Architecture

- **Frontend:** React 19, Vite, TailwindCSS v4, Zustand. Deployed as a web application on Vercel and a native Android App using Capacitor.
- **Backend:** Django 5.1, Django REST Framework (DRF), Python 3.11. Deployed on Render.
- **Real-time Communication:** Django Channels with WebSockets (and Redis as the channel layer).
- **Database:** SQLite for local development; PostgreSQL for production persistence.
- **Authentication:** JWT (JSON Web Tokens) using python-jose and bcrypt.

5. Workflow of the Project

- 1 **Registration & Profiling:** Users create an account and list the skills they can teach and the skills they wish to learn.
- 2 **Discovery:** Users utilize the AI Matchmaking tool to find compatible partners or browse the manual Marketplace.
- 3 **Coordination:** A user sends a session request. If accepted, a real-time WebSocket chat room is created for them to coordinate.
- 4 **Execution & Settlement:** Once the session is completed, Skill Credits are automatically transferred (deducted from the learner, added to the teacher).
- 5 **Rating:** The users rate each other, dynamically updating their platform reputation.

6. Process of the Project & Execution Environment

The application operates through decoupled frontend and backend processes:

- **Backend Execution:** Running `python manage.py runserver` initiates the ASGI/WSGI Django server on port 8000. It connects to the SQLite database (`db.sqlite3` located in the root backend directory) which securely stores all user records, skill vectors, wallets, and chat logs.
- **Frontend Execution:** Running `npm run dev` starts the Vite server on port 5173/5174. It serves the React application and proxies API requests to the Django backend.
- **Mobile App:** Changes compiled via `npm run build` are synced to the Android Studio wrapper via `npx cap sync android`, allowing the application to be built as a standalone APK.
- **State Management:** Client-side state (auth tokens, active chats) is held dynamically in memory using Zustand, while permanent authoritative data is entirely stored in the Django backend database.